

f risks involved in building a feature are prior to commitments being made and development starting. We are trying out a concept of a rotation of DRIs for spikes. Spikes will be a two week time period where one engineer DRI works exclusively on breaking down an issue/epic by asking questions to determine risk and complexity, creating proof of concepts, and writing up iteration plans for implementation.

The DRI will not be expected to produce MRs during the period they are on rotation, but instead, will be expected to produce issue artifacts at the end of the period, so that a first iteration can be worked on in the following milestone (either by the DRI or other engineers).

By the end of the spike, there should be documented acceptance criteria for a first iteration, that the internal parties (spike DRI, PM, UX, and EM) have all agreed to.

DRI rotation:

DRI	Start date	End date	Spike
---	---	---	---
Alexandru Croitor	2023-01-23	2023-02-03	Moving work items
Simon Knox	2023-02-06	2023-02-17	Frontend of work items at the group level
Heinrich Lee Yu	2023-02-20	2023-03-03	Group by work items
Coung Ngo	2023-03-13	2023-03-24	Group by work items ~frontend focused
Mario Celi	2023-03-27	2023-04-07	
Deepika Guliana	2023-04-10	2023-04-21	
Eulyeon Ko	2023-04-24	2023-05-05	

Collaboration between backend and frontend

Using the ~"backend complete" label

The ~"backend complete" label is added to issues with multiple specializations (usually backend and frontend) to indicate that the backend component is complete. Add this label when the backend work is functionally complete, merged and verified but frontend, or other, work is ongoing.

Picking something to work on

The Plan:Project Management Build board shows prioritized work with workflow columns relevant to implementation. There is an additional column to show in-progress community contributions. Filtering it by ~backend shows issues for backend engineers to work on.

It's OK to not take the top item if you are not confident you can solve it, but please post in splan if that's the case, as this probably means the issue should be better specified.

Queueing Experimentation

As part of our ongoing efforts to improve efficiency, we will experiment with queueing techniques instead of traditional weight/story points for the migration of legacy issues to

work items project. Starting in milestone 17.3, we will:

1. Refine Required Issues: Review the list of required issues for the first MVC.

- Determine the best way to group bundles of tasks, using issues or epics.
- Break down large issues into tasks, aiming for tasks that are small enough to be moved from ~"workflow::in dev" to ~"workflow::in review" within 5 business days.

2. Prioritize Tasks: Prioritize the tasks from step 1.

3. Update Workflow: Update the picking-something-to-work-on section to include the list of tasks, monitoring work in progress to keep it as low as possible.

This will help us manage and report progress more effectively using queue management principles.

High Severity Issues

```
{{% include "includes/engineering/plan/high-severity-items.md" %}}
```

Working on unscheduled issues

Everyone at GitLab has the freedom to manage their work as they see fit, because we measure impact, not activity. Part of this is the opportunity to work on items that aren't scheduled as part of the regular monthly release. This is mostly a reiteration of items elsewhere in the handbook, and it is here to make those explicit:

1. We expect people to be managers of one, and we use GitLab ourselves. If you see something that you think is important, you can request for it to be scheduled, or you can work on a proposal yourself, as long as you keep your other tasks in mind.
1. If you feel like you want to have some specific time set aside, but aren't interested in the topics of an existing event, feel free to label issues with "For Scheduling" and copy your manager for visibility.

When you pick something to work on, please:

1. Follow the standard workflow and assign it to yourself.
1. Share it in splan - if not even more widely (like in development or backend).

Retrospectives

GitLab milestones end on the 3rd Thursday of each month, and we run async retrospectives tied to that schedule. The retrospectives consist of three phases:

Phase 1: Collection

When:

Starts: 3 Mondays before milestone ends
Ends: End of release week (milestone Thursday)

What happens:

- Team submits retro topics via the Slack shortcut (·Project Management Retro Ideas)
- Weekly reminders go out
- Responses are sent to a google sheet that the facilitator has access to

Phase 2: Synthesis

When:

Starts: Friday of release week

Ends: Following Tuesday

What happens:

- Facilitator reviews and groups entries into themes
- Duplicates removed
- Poll generated to vote for the top two or three topics to discuss

Phase 3: Discussion & Action

When:

Starts: 1 week post-release (Tuesday·Friday)

What happens:

- Facilitator adds topics to generated retrospective issue
- Team reviews points async and comments on the items
- Team selects 2-3 action items to carry forward
- Owners are assigned and new issues are created to track follow-up

Example Timeline (Milestone 18.3 Ends 2025-07-17)

Phase	Dates
Collection	June 30 · July 17
Synthesis	July 18 · July 22
Discussion/Action	July 22 · July 25

Reminders

- Use the Slack shortcut to submit retro ideas anytime during the collection window.
- Weekly reminder messages will be posted to encourage participation.
- Keep it short · even one sentence helps.

Dashboards

{{< tableau height="600px" toolbar="hidden" src="https://us-west-

2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}

 {{< tableau/filters "GROUPLABEL"="project management" >}}
{{< /tableau >}}

 {{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
 {{< tableau/filters "GROUPLABEL"="project management" >}}
 {{< /tableau >}}

 {{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
 {{< tableau/filters "GROUPNAME"="project management" >}}
 {{< /tableau >}}

 {{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
 {{< tableau/filters "GROUPLABEL"="project management" >}}
 {{< /tableau >}}

Useful links

- Plan:Project Management Build board - this shows work in the current release
- splan in Slack
- Recorded meetings
- Retrospectives

SECTION: engineering/devops/dev/plan/working-groups/_index.md

SECTION: engineering/devops/dev/plan/working-groups/epic-work-items-migration.md

Epics to Work Items conversion plan and status

Attributes

|Property|Value|

|---|---|

|Date Created|2023-10-02|

|Target End Date| 2024-07-12 |

|Slack|splan, fwork-item, gproductplanning (only accessible from within the company)|

|Google Doc|Agenda (only accessible from within the company)|

|Epic|&9290|

|Overview & Status|See Exit Criteria Progress|

Context

Plan stage is developing Work Items as a platform to build GitLab features like Issues, Epics, Tasks, etc. to be more extensible to meet product needs. This working group is focused on the migration of Epics to Work Items and provides detailed insight on individuals involved in the effort, the technical plan for the migration, as well as current status and exit criteria.

Weekly Sync

WG members meet for sync call every Thursday at 14:30 UTC to discuss the actions taken since last call as well as steps for next week, meeting is recorded and publicly shared in GitLab Unfiltered YouTube channel.

`<iframe width="560" height="315" src="https://www.youtube.com/embed/videoseries?list=PL05JrBw4t0KrEtDK8xRruSY5VtQb9DmdX" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>`

Exit Criteria

Updates are being tracked in &11777 (updated every Monday).

This Working Group has the following exit criteria:

1. A working Epic Work Item Type is available to all customers running the latest version of GitLab.
1. The implementation does not create significant scaling or saturation risks for at least 6 months, and does not block feature development.
1. A plan is in place to clean up technical debt within 6 months.

Stage 1 - Achieving Feature Parity between Epics and Work Items

Task Progress DRI (group or individual)
--- --- ---
Add native support for work items at group-level 100% Project Management
Add support for relating work items based on different relationship types 100% Product Planning
Add support for relating legacy issues as children to work items 100% Product Planning
Add support for cross-group hierarchies in Hierarchy widget 100% Product Planning
Start and Due date inheritance support in Work Items 100% Kassio Borges (BE) & Himanshu Kapoor (FE) - Knowledge
Roll-up statistics around children count, weight sum, and health status in Hierarchy widget 100% Project Management
Add support for Ancestor widgets 100% Product Planning

Stage 2 - Epics data migration plan

Task Progress DRI (group or individual)
--- --- ---
Block widgets which will be unavailable for migrated Epics 100% Product Planning

Setup synchronization between Epics table and Work Items table in database	100%	Product Planning
Setup synchronization of widgets data between Epics and Work Items along with handling change in Epic Work Item ID	100%	Product Planning
Introduce Work Item Epic creation ability at Group level	100%	Product Planning

Stage 3 - Post-migration roll-out preparation

Task	Progress
Ensure REST API interoperability with migrated Epics	100%
Determine migration strategy for unique ID and IID of migrating epics to ensure that stale reference redirect	100%
Database cleanup for stale columns and implementation logic of legacy Epics	80%

Roles and Responsibilities

Working Group Role	Work Stream/Specialism(s)	Person	Title
Facilitator & Functional Lead	Backfilling (3)	Alexandru Croitor	Senior Backend Engineer, Plan:Project Management
Functional Lead	Q4 Facilitator	Kushal Pandya	Fullstack Engineering Manager, Plan:Product Planning
Functional Lead	Feature Parity (1)	Eugenia Grief	Senior Backend Engineer, Plan:Product Planning
Functional Lead	Syncing (2)	Felipe Artur	Senior Backend Engineer, Plan:Optimize
Functional Lead	Delegation (4)	Nicolas Dular	Staff Backend Engineer, Plan:Product Planning
Functional Lead	Create legacy epics for WI epics (5)	Mario Celi	Backend Engineer, Plan:Project Management
Functional Lead	Customer Pilot (6)	Heinrich Lee Yu	Staff Backend Engineer, Plan:Project Management
Member		Kassio Borges	Staff Backend Engineer, Plan:Knowledge
Member		Himanshu Kapoor	Senior Frontend Engineer, Plan:Knowledge
Member		Adam Hegyi	Staff Backend Engineer, Plan:Optimize
Member		Coung Ngo	Senior Frontend Engineer, Plan:Project Management
Member		Florie Guibert	Senior Frontend Engineer, Plan:Product Planning
Member		Rajan Mistry	Frontend Engineer, Plan:Product Planning
Member		Abhilash Kotte	Sr Full Stack Engineer, Plan:Product Planning
Member		Donald Cook	Fullstack Engineering Manager, Plan:Project Management
Member		Gabe Weaver	Senior Product Manager, Plan:Project Management
Member		Amanda Rueda	Senior Product Manager, Plan:Product Planning
Member		Jacki Bauer	Product Design Manager, Plan

SECTION: engineering/devops/dev/plan/working-groups/work-items-api-performance.md

Attributes

| Property | Value

Date Created	2025-03-03	
Target End Date	2025-08-29	
Slack	wgwork-items-api-performance	
Google Doc	Agenda	
Epic	https://gitlab.com/groups/gitlab-org/-/epics/16919	
Overview & Status	See Exit Criteria Progress	

Context

Many Plan features depend on the same work-items APIs, database and search architecture:

- issue searches for projects and groups
- boards
- GLQL

These features have similar performance problems:

- Relatively simple search queries on the large groups currently time-out.
- GLQL queries timing out

Performance of these APIs also affects our error budgets.

Weekly Sync

TBD

Exit Criteria

- [] Migrate issue filters for groups/projects to Work Items API
- [] Measure performance of the Work Items API and set improvement targets, such as:
 - Specific common queries not timing-out
 - Get X-th request duration percentile under X target
 - Get Error-budget in the green
- [] Reach targets set on the previous step

Roles and Responsibilities

Working Group	Role	Person	Title
Facilitator		Eugenia Grief	Senior Backend Engineer - Plan::Product Planning

Member	Alisa Frunza	Backend Engineer - Plan::Knowledge
Member	Heinrich Lee Yu	Staff Backend Engineer - Plan::Project Management
Member	Kassio Borges	Staff Backend Engineer - Plan::Product Planning
Member	Alexandru Croitor	Senior Backend Engineer - Plan:Project Management
Member	Nicolas Dular	Staff Backend Engineer - Plan::Product Planning
Member	Dmitry Gruzd	Staff Backend Engineer - Foundations::Global Search

SECTION: engineering/devops/ops/_index.md

The Ops Sub-department is composed of development teams working on Verify, Package, Deploy, Service Management and Observability features of GitLab DevOps Platform.

Teams and Handbook Pages

The following teams comprise the Ops sub-department:

- Verify stage - handbook
 - Pipeline Execution - handbook, jtbd
 - Pipeline Authoring - handbook, jtbd
 - Runner - handbook, jtbd
 - CI Platform - handbook
- Package stage - handbook, jtbd
 - Container Registry - handbook
 - Package Registry - handbook
- Deploy stage
 - Environments - handbook, jtbd

Product Direction

Teams in the Ops Sub-Department map to several Product Sections. Product direction can be found on the following direction pages:

- CI Section Direction
- CD Section Direction

Slack channels

- ci-section - CI Section discussions spanning the Verify, Package and Monitor Stages
- cd-section - CD Section discussions spanning Deploy Stage
- opsstaffplus - Foster collaboration and technical leadership within Ops section
- doe-ops - Channel for GitLab's Director of Engineering, Ops
- ops-staff-confidential - Channel for reviewing topics for Ops Engineering leaders. Please

use do-ops or other public channels for topics that don't require confidentiality.

Meetings

Zoom Recordings synced to Google Drive

GitLab encourages transparency by default, and when meetings are recorded, they can be automatically synced to Google Drive as highlighted in the section about how we conduct video calls at GitLab. To summarize:

1. Record meetings to the cloud in Zoom, and add the text [REC] anywhere in the meeting title (e.g. Google calendar event)
1. It will automatically be added to the GitLab Videos Recorded folder on Google Drive, with a naming convention of your email-Meeting Title.
1. After a meeting ends, Zoom may take some time to process the recording before it is actually available. The sync to Google Drive happens on the hour mark, so if the recording is not available, it may take another hour to be transferred.
1. Future recordings of recurring meetings can be found in the same folder your email-Meeting Title, even if alternate hosts start the recording.

Ops Engineering Showcase

Starting in 2023 in FY24-Q1, a fresh new format for this showcase will include the recording and sharing of videos that team members have created. Once a video has been created, an async discussion issue should be created using this template and listed under the relevant quarterly epic. Based on the amount of questions being discussed in the issue, an AMA may be set up to discuss further in a group setting.

Like before, these videos are intended to be opportunities for engineers to show off the things they've been working on.

Beginning in March 2023 all Ops teams are required to provide at least one showcase video and async discussion issue per month. We require participation in this process because it helps reinforce our values of Results, Transparency, and Iteration. In addition, these showcases are valuable in helping team members across the company better understand the progress and obstacles across the many product areas Ops team members are focused on. They are also fun to watch. These videos will be reviewed monthly by the Director of Engineering and other leaders in the group. The team's Engineering Manager is the DRI for submitting the showcase, though in many cases the demo may be delegated to another member of the team.

Team members are encouraged to use these videos to transparently showcase the current state of their projects. It is not encouraged to spend significant time prepping for these demos or only showcase finished, polished functionality; showing functionality in an incomplete state is encouraged. Videos may be specifically recorded for this purpose or be excerpts from team meetings, etc. Some suggested ideas to consider could be related to error budget improvements, performance improvements to current features, refactoring to codebase for future maintainability, etc.

Ops Engineering Showcases

Recordings

<details markdown="1">

<summary markdown="span">Past Recordings</summary>

Quarterly Recordings

NOTE: - changed to monthly recordings in FY24-Q3.

Year	Quarter	Links
-----	-----	-----
2023	FY24-Q1	November, December, January
2023	FY24-Q3	August, September, October
2023	FY24-Q2	Videos
2023	FY24-Q1	Videos

Monthly Recordings

Year	Month	AMEA	APAC	Async videos
-----	-----	-----	-----	-----
2022	December	No showcases	No showcases	
2022	November	No showcases	No showcases	
2022	October	Split GitLab Monolith into components	No showcases	TaskScaler 101
2022	September	Import packages with a CI pipeline	No showcases	
2022	August	No showcases	Async video ->	Leader election in GitLab Agent for Kubernetes
2022	July	Video	No showcases	MR Coverage Report & CI Workflows
2022	June	Video	Video	Demo
2022	May	Video	No showcases	
2022	April	Video	Video	
2022	March	Video	No showcases	
2022	February	Video	No showcases	
2022	January	Video	No showcases	Jobs UI Refactor
2021	December	Video	No showcases	
2021	November	Video	No showcases	
2021	October	Video	Video	
2020	November	Video	Video	
2020	October	Video	Skipped	
2020	September	Video	Video	
2020	August	Video	Video	

</details>

Hackathons

Hackathons provide a great opportunity for team members to experiment with working on ideas as possible features that can be integrated into GitLab in the future. The intent is to inspire creativity and encourage collaboration amongst those you may not have the opportunity to work with on a regular basis, set aside regular milestone deliverables, and have some fun with tackling different challenges.

With the IT landscape rapidly changing with explorations in emerging technologies, having Hackathons take place twice a year offers everyone a chance to contribute to different areas of interest.

Planning Processes

Planning processes followed by teams in the Sub-department:

- General Ops Sub-department Planning Process

Forced Priority planning/scheduling

Our prioritization framework describes a number of Forced Priority labels that present a high risk to our customers and our business. As such, it's critical that we complete this work within the appropriate SLO/SLAs where possible. An example of this are security issues, here's a dashboard for past due and soon to be due security issues: <https://10az.online.tableau.com/t/gitlab/views/DRAFTIssueTypesDetail/PastDueIssueList/d1401498-2050-4321-b0f4-ab51047fbef1/1909fa14-fdf0-4682-8a07-f02d8c8d4803>

With this in mind, here's a process that groups are encouraged to use and iterate on in the event that they don't have an existing approach or are falling behind on SLA/SLOs.

To be clear, this is not a required process, if your group's approach is working, please continue with that! Ownership of scheduling forced priority work falls on our whole team, not a particular person.

1. When forced priority issues are assigned to your group, schedule those issues right away regardless of severity label
1. Have the team review the issue to make sure there's a viable solution and plan (weight the issue)
1. Have someone in the group assign themselves to work on the issue
1. Update the issue regularly with progress
1. Treat blockers with the same or higher priority than the original issue

Sustaining / Non-Sustaining planning exercise

In FY23, each EM in Ops was asked to participate in a capacity planning exercise that involved evaluating the sustaining vs non-sustaining work in their team's backlog. This exercise was also repeated by the Verify Pipeline teams in FY24 to better understand Engineering capacity needs for each of the Pipeline teams.

Each team created a staffing justification based on the breakdown of sustaining vs non-sustaining issues in their backlog. EMs estimated the number of backend engineers and frontend engineers that would be needed, per quarter, of the given fiscal year to burndown these issues, as ordered by the prioritization framework and relevant priority/severity labels.

Sustaining work

Sustaining work refers to the most important security, availability, scalability and performance, essential for "keeping the lights on" for the feature categories the team

supports. Capacity planning for sustaining work was based on meeting the SLO/SLAs of these types of issues. The labels for sustaining work include, but are not limited to:

- bug::vulnerability (note: specifically this label, as an issue with only a security label does not make it a security vulnerability)
- bug::availability
- bug::performance
- infradev

Non-Sustaining work

Non-sustaining effort refers to all other work the teams would work on, including:

- type::feature
- type::maintenance (e.g. technical debt that does not impact availability, scalability, performance, etc)
- all other type::bug issues not covered as sustaining work
- reviewing community contributions
- responding to customer inquiries / support requests

Ops Promotion Process

Tips for writing Promotion Documents

- The format and guidelines are as listed in this Google Doc template
- Results is our most important value to cover in promotion docs, especially at Staff+ levels. This should be short (making it succinct can be easier to understand), but descriptive enough to show the impact these results have had.
- Impact should also be highlighted (quantifiable where possible), with examples such as:
 - customer-facing impact (CMAU, ARR growth, retention, adoption, etc)
 - improvements in efficiency, team process or other team metrics
- Examples demonstrating leadership, ownership and accountability (going above and beyond for your current role):
 - Your role on a project or initiative ("how did you successfully help deliver X?", "what role did you play in this Working Group?")
 - MR Review efforts (your role as a maintainer for your team or for the rest of the department in your domain)
 - Collaborating with your team's stable counterparts
 - Mentoring/coaching others on your team or outside your team (e.g. participation in mentorship programs, coaching in MRs as a reviewer/maintainer)
 - Influencing your team's technical direction or contributions to more strategic discussions
 - For Staff+ engineers: this could mean authoring or contributing to architectural design documents or leading technical discussions with your team or driving cross-stage collaboration
 - Involvement in recruiting efforts (e.g. as an interviewer and/or reviewer of the technical assessment)
- Examples of promotion documents from previously approved Ops promotions can be found in this Google Drive folder

Promotion Document Reviews

1. Team Member and Manager work together to create promotion document.
1. Promotion document is shared with Ops Sub-Department managers in ops-staff-confidential channel (private to Ops Managers, Senior Managers, Director) to receive feedback on the document content and how it calibrates with others in the Sub-Department. This can be done early and often.
1. Manager requests another Manager in the Sub-Department review and approve the document.
1. Manager requests Senior Manager in the Sub-Department review and approve the document.
1. Manager requests Director add Team Member to list of Sub-Department planned promotions. If the team member cannot be added Director will provide feedback to Manager as to why.
1. Director presents promotion doc at Development Department quarterly promotion review meeting. If the promotion is not approved Director will provide feedback to Manager as to why.

Quarterly Promotion Review Meeting

Ops Managers, Sr. Managers, and Directors meet in the last month of each quarter to:

1. Review promotion projections for each group. Managers add a list of their direct reports who they anticipate will be ready for promotion in the next 3 quarters to the meeting agenda. Note these lists may change and are not a guarantee a team member will be promoted in a specific quarter. The lists are used for planning purposes, to help us anticipate the number of team members approaching promotion, budget impact, and to support calibration across the group.

These lists should include:

1. Team Member name
1. Targeted promotion quarter
1. Link to promotion doc
1. Whether the doc has received any Ops Manager or Ops Senior Manager approvals.
1. Review Ops team member promotion docs planned to be presented at Development Department quarterly promotion review meeting.

Promotion Mentor for Staff+ Roles

Team members who aspire to advance within our organization are advised to identify a mentor in the targeted role, and reach out to them about a mentorship. Involving a mentor in the peer promotion review process is beneficial, as it ensures:

1. The feedback is informed by comprehensive understanding and relevance to the individual's development.
1. The feedback, being familiar and constructive, will not catch the mentee off guard and will provide clear directions for improvement from peers they'd work with directly.

Individuals pursuing promotion, possibly in consultation with their managers, should maintain regular communication with their mentor to evaluate progress and explore further opportunities for growth. This initiative also aims to cultivate leadership qualities among our senior Staff+ engineers.

We plan to trial this optional process in Ops during Q1FY25. Progress and established mentor/mentee pairs can be found in this epic

Ops Hiring process

We have limited capacity in the recruiting team so hiring managers are experimenting with the Ops Hiring process. We also hold a weekly hiring manager sync meeting (see Meetings section).

Objectives & Key Results (OKRs)

Creating OKRs

- Target a 70% completion rate for OKRs so that we are setting goals ambitiously. They should be challenging enough that accomplishing 100% is a stretch goal, since GitLab considers OKRs to be stretch goals by default.
- If your OKRs are associated with any specific types of issues, consider reviewing the prioritization framework, and align with your team on your OKRs priority relative to one another (for example, OKRs with issues with "forced prioritization") and be clear on planning of the associated work over the quarter.
- Ensure Key Results (KRs) are measurable, by specifying a number (% or unit) as the target. Be clear on the starting number, and the expected end result (e.g. from x to y).
- Ideally create no more than 9 KRs per quarter, ideally with approximately 3 KR per objective.

Tracking and completing OKRs

- Tracking quarterly OKRs for Ops Engineering groups.
- DRIs are expected to score OKRs on a regular basis, at least monthly, if possible.
- At the end of the quarter, DRIs can add a retrospective as a comment to reflect on the progress of their KR, in the following format:

markdown

Good

- Things that went well...

Bad

- Things that didn't go so well...

Try

- What might you do differently next time?

Key Projects

For projects that span many milestones we are experimenting with additional project management processes.

Key projects will be listed in the table below:

Project Name	DRI	Tech Leads
----- ----- -----		
----- ----- -----		
CI Data Partitioning	Project plan	
Caroline Simpson	Marius Bobin	

CI Primary Key Conversion	Project plan
Drew Stachon / Caroline Simpson Tianwen Chen	
Container Registry v2 Self-Managed	Project plan
Crystal Poole Hayley Swimelar	
CI Secrets Management	Project plan
Scott Hampton Albert Salim	
Cluster Web Terminal	Project plan
Nicolò Maria Mezzopera	
Kubernetes Dashboard	Project plan
Nicolò Maria Mezzopera	
Autoscaling for GitLab Runner	Project Plan Nicole Williams
Dedicated SaaS Runners For GitLab Dedicated	Project Plan Nicole Williams
CI Steps	Project plan
Kamil Trzeciński / Cheryl Li Joe Burnett	

Completed Projects

Project Name	Link to Project Plan
DRI Tech Leads	Completed
-----	-----
-----	-----
-----	-----
CI/CD Catalog	Project plan
Mark Nuzzo Laura Montemayor and Avielle Wolfe FY25-Q1	
Merge Train Improvements (Fast-Foward Support)	Project plan Caroline Simpson Allison Browne, Hordur Yngvason FY24-Q4
O11y Distributed Tracing	Nicholas Klick FY24-Q4
GCP Secrets Integration	Project plan Scott Hampton Albert Salim FY24-Q4

Key Project Planning

Projects which are listed as key projects should create these project planning artifacts:

Project Plan in the handbook under `/engineering/devops/ops/project-plans/<projectname>/index.html.md.erb`

Since the project plan is stored in the handbook it is easy to track changes in the project plan and see how it evolves over time.

Project plans should (roughly) follow this format (from <https://www.rubick.com/weekly-project-plans/>):

text

Weekly project plan template

Week of Jan 4th

Single chart shows up in Slack. Data is canned.

Schedule risk: we're validating our list of chart types are all technically feasible. We'll demo outcome of that investigation.

Week of Jan 11th

Chart data reflects live information, and is functional in Slack chart.

Additional chart type shows in Slack room, with most basic visual design.

We've shown to at least one alpha customer for feedback. We start sharing with them every week from here on out.

Jessica is on-call and doing interrupt-driven work for week.

Week of Jan 18th

Most important feedback from alpha customers incorporated. Other work is prioritized to future milestones.

Last chart type added.

Week of Jan 25th

Holiday Jan 26th.

Charts look great and are thoroughly tested, instrumented. We'll show usage dashboards.

Release end of week.

Updates to the plan should be made weekly, marking weekly items done and making updates based on current information. Authors should merge updates to their project plans without review/approval.

We expect the use of project plans to reduce the amount of EM reporting by serving as a SSOT for project execution planning. For example this info no longer needs to be included in async updates (replace with a link to project plan).

Key Project Review Meetings

We hold periodic sync meetings to review key projects and collaborate on the plans for these important deliverables.

Key Project DRIs should prepare for the meeting by creating a Key Project Review Prep Issue. This should be completed and shared at least 48 hours in advance of the meeting to allow time for other team members to review the content.

The objective is to keep these meetings focused on execution, technical planning, and accountability within the Engineering group. These are open to anyone who wishes to attend. When it's necessary to involve a broader group in some cases, we can plan additional follow ups case by case.

We aim to meet synchronously where possible given the benefits of being able to talk through the project in a higher bandwidth context. Rescheduling is preferred if the proposed dates are

not feasible for attendees. We could re-evaluate whether we can achieve the same goals with async after we have more experience running the process.

The focus of the meeting time will be on topics such as:

1. Do we all understand the goals, plan, and timeline?
1. Obstacles/risks and mitigation plans
 1. Unmet dependencies
 1. Staffing levels
 1. Competing priorities
 1. How to unblock and/or accelerate
1. Recognizing progress and key team member contributions

Async Updates (No Status In Meetings)

We have a policy in Ops to keep status updates out of meetings. Instead of reporting on status in meetings Directors, Senior Engineering Managers, Engineering Managers and Principal Engineers provide regular async updates.

The content of these updates varies by individual and role:

Async Update Guidance

Engineering Manager

1. Cadence: Weekly to Monthly (Optional)
1. Topics:
 1. Highlights & Accomplishments
 1. Team Projects & Priorities (progress, blockers, key decisions, etc.)
 1. What's next - what to expect in next update

Senior Engineering Manager

1. Cadence: Monthly (Optional)
1. Topics:
 1. Milestone Review
 1. Allocation of focus in milestone plans (major features, maintenance, etc.)
 1. What tradeoffs or decisions should be made prior to Milestone kickoff? For example:
 1. Is the team overcommitted?
 1. Does the allocation of effort need to shift to meet specific goals? (e.g.

Security SLAs)

Principal Engineer

1. Cadence: Weekly (Optional)
1. Topics:
 1. Highlights & Accomplishments
 1. Projects & Priorities (progress, blockers, key decisions, etc.)

To create a weekly async update

1. Create a GitLab issue in any project.
2. Apply the label ~OpsSection::AsyncUpdate. (The label ~OpsSection::Weekly-Update can also be used for backwards compatibility reasons.)
3. Choose a descriptive title. Include your update in the issue description.

To HIGHLIGHT an item for broader sharing

1. Begin any paragraph in your update with the text HIGHLIGHT:
1. These paragraphs will automatically be copied into the Highlights & Accomplishments section of the monthly Director's update.

To review Ops weekly async updates

1. All issues with the ~OpsSection::AsyncUpdate or ~OpsSection::Weekly-Update label are summarized in weekly issues in the Ops Status Updates project.
1. All team members are encouraged to comment and ask questions in weekly async update issues. This provides useful feedback to the author and opportunities to collaborate.
1. Director reviews all updates weekly (from the prior week) and provides feedback.
1. Senior EMs reviews all updates from their teams weekly (from the prior week) and provides feedback.

Benefits of reporting and reviewing status async vs. synchronously in meetings

1. Collaboration - By sharing announcements, achievements, goals, and plans we are empowering team members with the information they need to make sound decisions and succeed in their roles.
1. Results - Async updates provide an important channel for sharing accomplishments and progress.
1. Efficiency - Expecting each team member to poll for the status they need is extremely inefficient and becomes more inefficient as the organization grows. By using a Publish Subscribe model we are able to share information much more efficiently.
1. Diversity, Inclusion & Belonging - Async status is more inclusive of team members across timezones. Not every team member will be able to make a sync meeting (due to timezone or other factors) but everyone can participate in an async update.
1. Iteration - Taking the time to checkpoint where we are at each week prompts the author and readers to consider ways to iterate and to reflect on incremental progress.
1. Transparency - Sharing information in a consistent, accessible way increases transparency and reduces the threshold for team members to contribute.

Escalations from Support Team

We are piloting a process to make coordinating with the Support team on customer escalations more efficient.

See [How to Use GitLab.com to Formally Request Help from the GitLab Ops Development Team](#) for process details.

If you encounter an Ops-related customer escalation, that seems to have a high customer priority, and would benefit from additional visibility, please post a link with a short description in [doe-ops Slack channel](#).

Supporting Customers on Dedicated instances

When teams are asked to triage support requests for customers on GitLab Dedicated and engineers need access to their logs, a sync can be arranged between the engineer and the assigned Support Engineer for a screen sharing session.

However, when further troubleshooting is needed by the engineering team or async collaboration is preferred, follow these steps to request access:

1. Create an access request (AR) and list the engineers who require access. (Note that EMs can create this issue and provide Manager Approval on behalf of the engineers)
1. Assign the AR to the EM and PM of the Dedicated group for read-only log access.
1. Reference the AR issue in the support tracker issue so that the Support Engineer is aware.

SECTION: engineering/devops/ops/general-planning.md

Groups within the Ops Sub-department follow an async planning process to coordinate planning and discussion between members of the group. This process is related to the general GitLab Product Development Timeline process and adds additional practices we've found useful.

The goals of this process are to ensure we:

Transparently describe the goals for each milestone so team members can align on them and effectively plan and execute their contributions.

Efficiently coordinate handoffs between the validate and build tracks within each milestone, such as design and development.

Efficiently discuss and define the scope of work for development, product design, and quality engineering.

Teams can customize this process to meet needs specific to their team as long as the customizations are documented on their team handbook page.

1 month before start of milestone

PM: create a Planning Issue for the milestone from the team's Planning Issue Template.

PM: Set the Planning Issue milestone based on the current working milestone (e.g. Planning Issue 12.10 goes in milestone 12.9).

PM: update Goals for the Milestone in Planning Issue description.

PM: assign the Planning Issue to the PM, Engineering Managers, Product Designer, and Software Engineer in Test.

PM: ask for feedback on the Goals for the Milestone by adding comments for the product group members and other teams' group members if cross-team collaboration is required.

~3 weeks before start of milestone

Engineering Managers: update Scope of Engineering Work

Product Designer: update Scope of Work for UX

PM: review scope of works and provide feedback to Engineering Managers and Product Designers via comments on the issue. Synchronous conversations are encouraged but should be summarized

as comments.

~2 weeks before start of milestone

Product Designer: attach first draft designs to the planning issue (directly or via linked issue)

Product Designer: add a comment asking for feedback on first draft designs

Engineering Managers: add a comment asking for feedback on Scope of Engineering work from engineers or other GitLab team members

~1 week before start of milestone

Product Designer: attach next draft designs to the planning issue (directly or via linked issue)

Product Designer: add a comment asking for feedback on next draft designs

Software Engineer in Test: review issues under Scope of Engineering Work and apply ~quad-planning label

Software Engineer in Test: update Scope of Work for Testing

Release day

PM: close Planning Issue

Planning Issue Template

Planning issue templates follow a format similar to this. Teams can add additional headings specific to their team.

markdown

<Section> Planning Board

<!-- link to your sections planning board -->

Goals for the milestone:

<!-- Replace with the high-level goals to be achieved by the end of the milestone -->

Goal 1...

Goal 2...

Scope of Work for Engineering

<!--

List of issues in priority order go in this table

Use :whitecheckmark: for Frontend and/or Backend if there is work for that team

-->

Priority	Issue	Category	Notes	Frontend	Backend
1					
2					

Scope of Work for Engineering Debt

<!-- What work of Engineering Debt from the past will Engineering be focused on?

- Should be at least 1 Issue per Milestone
- The more we can handle the better... balance

-->

| Issue | When it should be ready |

|-----|-----|

| | |

Scope of New Work for UX

<!-- What new work will UX be focused on?

- Design based on research in previous Milestones
- Research to help inform designs for future Milestones
- Research to validate previously designed UI

-->

| Issue | When it should be ready |

|-----|-----|

| | |

Scope of Work for Deferred UX

<!-- What work of Deferred UX from the past will UX be focused on?

- Should be at least 1 Issue per Milestone
- The more we can handle the better... balance

-->

| Issue | When it should be ready |

|-----|-----|

| | |

Scope of Work for Software Engineers in Test

<!-- What work will Testing be focused on? -->

| Issue | Investigates/Tests | Due on |

|-----|-----|-----|

| | | |

/label <!-- add section label here --> ~"Planning Issue"

Team Planning Issue Templates:

Environments Team Template

Monitor APM Team Template

Monitor Health Team Template

Planning Issue Board

All the planning issues are viewable on the Ops Section - Issue Planning (Issue Board)

SECTION: engineering/devops/ops/hiring-process.md

We are running with some limited capacity in the recruiting team and we are asking hiring managers to cover all steps in the hiring process. Please prioritize your time in the following order when spending time on recruiting:

1. References & Justifications
2. Filling in scorecards/feedback
3. Sourcing/Reviewing applications

Hiring Process

The process for Ops is a mix of the current hiring manager (HM) tasks and the talent acquisition (TA) tasks. We use Evergreen Requisitions as a single pipeline for inbound applications to similar roles. In addition, each hiring manager will use a separate Greenhouse requisition for their specific role. The Evergreen Req will be used in conjunction with the team-specific req to track candidates end to end through the hiring process.

Step 1. Identify hiring need

Identify hiring need

1. Work with Product Manager as they are DRI of headcount planning, Finance, and Talent Acquisition to include vacancy in the hiring plan.
1. Create or Review the Job Family

Step 2. Kickoff

1. Create Kickoff

1. Request a GitLab Hiring Plan ID (GHPID) from Director of Engineering, Ops (@sgoldstein)
1. Once the GHPID has been requested the assigned recruiter will create a kickoff issue in the gl-talent-acquisition/req-intake project and mention you on it. Once the kickoff issue is created confirm that:

1. There is a separate kickoff issue for each req you are hiring (each unique GHPID). For identical roles, one kickoff issue can just be a link to another, but having a separate issue for each req is valuable for tracking purposes.

1. The kickoff issue has the ~section::ops label applied so we can track in this board (confidential) and use it as the single source of truth (SSOT)

1. If you have not received a kickoff issue within 3 business days of requesting the GHPID escalate this by mentioning @sgoldstein in the doe-ops slack channel.

1. Complete Kickoff

1. Add the new vacancy to the handbook <https://gitlab.com/gitlab-com/www-gitlab-com/-/tree/master/data/teammembers/vacancy>

Step 3. Setup job in Greenhouse

1. Work with TA to open the vacancy in Greenhouse

1. Setup scoreboard

1. Publish job ad internally. External job ads will be posted from the corresponding Evergreen requisition.

1. [Optional] Add additional questions to the application, such as number of years of Rails experience, or qualities relevant to your role

1. Add your own manager as a hiring manager in the req

1. Job Setup (tab) > Hiring Team (left nav)

1. Click Edit under Hiring Managers and add your manager

1. If your manager's name doesn't show up, it's likely that they don't have any permissions to view this role currently.

1. Scroll down to the section Who can see this job? and grant your manager Job Admin: Hiring Manager permissions

1. Return to the previous step, and now you can edit the list of Hiring Managers and add them

1. Ensure you have hiring manager access to the Evergreen Requisition for your role

Step 4. Reviewing Candidates

Reviewing your Requisitions

1. Review applications in the requisition you are the hiring manager for.

1. If you are interested in a candidate, move them into the Qualified stage, where the TA will also review prior to setting up a screen.

1. :warning: TA is trialing Greenhouse automation with some requisitions where candidates automatically receive an email to book a screen with them when they're moved into Screening. Please confirm with TA whether this automation is set up for your role.

Reviewing Evergreen Requisitions

1. Review applications in the applicable Evergreen requisition

1. If you are interested in a candidate, move them into the Qualified stage and tag your TA to request a screening (see Step 6)

1. Note: Candidates will stay in the Evergreen requisition until they have completed screening and technical interviews, at which point they will be transferred to team-specific requisitions.

1. Do not reject applications in the Evergreen requisition unless the applicant's location is a country where hiring is restricted

Step 5. Sourcing

In addition to reviewing candidates in the Evergreen req, you may also wish to do sourcing for your team-specific role. The main purpose of sourcing is to identify and engage top talent.

1. Get a recruiter LinkedIn account and setup

1. Setup LinkedIn for sourcing, for example:

1. Create your own templates

1. Create projects for each of your open positions or make sure you are invited to

1. Search for candidates

1. If you are getting support from a sourcer, follow the steps to review inbound applications

1. Reach Out. Consider using the templates listed in the resources. Make sure to ask for contact details of the prospect, you will need this in the next step.

1. Add prospects to your team-specific requisition in Greenhouse. In the prospect's profile, click in the three dots, and click on export to ATS.

1. After importing to Greenhouse, edit the profile (under Details tab, there is an edit button) to include the email and any other contact information. This is important because Greenhouse automatically sends an email to the candidate when is moved from prospect to candidate to fill a form which includes the CV to be uploaded there.

Resources:

- Mark Deubel's podcast: Foundations - Sourcing Outreach and Guest Show - Sourcing outreach
- Talent Acquisition Reach out templates (private)
- Ops reach out templates (private)
- Sourcing
- LinkedIn training (private)
- LinkedIn boolean search (private)
- Diversity Boolean strings (private)
- Single Search Permutator (private)
- Video training for Diversity Sourcing (private)

Step 6. Screening

To speed up scheduling screening calls, integrate Calendly with Greenhouse following these instructions. To then use this integration, move the candidate to the 'Screening' stage and select "Schedule with Calendly". There will be a template that will automatically populate and you select your Calendly event in the drop down.

1. Schedule screening calls (3-6 per week)

1. Tag TA on the candidates Greenhouse profile so they can support with screening calls. and work with TA in case they have capacity to do the screening

1. Screening. Don't forget to introduce the TA to the prospect at the start of the team interview stage so they can build a relationship with the candidate and ensure a consistent experience.

Step 7. Continuous check-in

Continuous check-in

Step 8. Schedule interviews

You have two options to Schedule interviews. You can either:

1. Tag TA on the candidates profile and they will organize everything

2. Start the process yourself. To do this:

1. Move the candidate to the Team Interview stage

2. Select Request Availability and search for either the "Ruby" or "Golang" technical

interview template. This is important as it has specific instructions

3. Then select Email the Team, choose the CES Scheduling Request Form

4. In the template, add - 'Technical Interview - Ruby or Go, Ruby or Go interviewers, 90 minutes' under details

5. x the Confirmed box to confirm you've requested their availability

6. Send the template and tag TA on the candidates profile to introduce themselves to the candidate

Step 9. Perform interviews

Step 10. Complete feedback

Complete feedback. Ideally the feedback is submitted within 1-2 business days of the interview. If you're unable to submit it within that timeframe, please reach out to the Hiring Manager and/or Recruiter with any early evaluation and when they can expect your feedback by.

Note: There are times when candidates who may not be a good fit in one role could be an ideal fit in another role. In those cases, please note that to the recruiter or share in Slack (i.e. ops-hiring, dev-hiring-mgmt, etc.) as it may be helpful for other hiring managers to be aware of to evaluate these candidates.

Step 11. References

1. Request references

1. Complete references

Step 12. Justification

1. Hiring Manager to complete Justification scorecard.

1. Justification scorecard will be reviewed by Director of Ops (or a delegate) for these criteria (see Engineering Hiring Process README for additional detail):

1. · Two "Strong Yes" scorecard ratings minimum.

1. All must-haves assessed and positive. These should be checked in the Justification scorecard. The ratings should reflect the hiring manager's evaluation of the candidate based on all feedback. They are not an average of scores given by the interview panel.

1. · Majority (e.g. 5 out of 8) of nice-to-haves assessed and positive in Justification. These should be checked in the Justification scorecard. The ratings should reflect the hiring manager's evaluation of the candidate based on all feedback. They are not an average of scores given by the interview panel.

1. · Negative scorecards (No, Definitely Not) from interviewers should be explicitly addressed with a brief comment in the Justification.

1. · Any previous GitLab job application rejections must be explicitly addressed in Justification. (Confirm with TA)

1. · Any red flag must be explicitly addressed in Justification.

1. · Each flag should have a corresponding item in the "How do we intend on setting this candidate up for success?" section to explain how the Hiring Manager plans to mitigate the flag.

<details>

<summary>Example justification structure with expected criteria</summary>

In what specific way(s) does this candidate make the team better?

- Describe each must have category for the role with a summary of how the candidate is a good fit.

What flags were raised during the interview process?

- If the candidate was previously rejected, determine the reason they were rejected and describe why and how they are a good fit for the current role. Your recruiter, Senior EM, or Director can help you determine the reason they were previously reject (you may not have access to the other roles in Greenhouse). In cases where they were rejected by another hiring manager who is still employed at GitLab it is good to follow up with that HM to get their input. Include that information in the discussion of why they are a good fit for the current role.

- If the candidate had a Thumbs-down scorecard, explain why and elaborate on the reason you, as the HM are still wanting to move forward with the offer to hire them.

- Do not list every possible flag, but rather just the important red flags to consider when justifying they are a good hire choice.

How do we intend on setting this candidate up for success?

- Expand on each flag described above and how you, as the HM plan to mitigate that flag and set the candidate up for success at GitLab.

Is the offer in-plan, and why? (e.g.: critical budgeted hire, backfill, transfer)

- Stating which kind of hire is sufficient, per the referenced examples.

Does the candidate meet a simple majority of nice-to-have requirements (5 of 9) for the role?

- Yes, [First Name] met a majority of the nice-to-have requirements.

</details>

SECTION: engineering/devops/ops/quarterly-okrs.md

FY24 Q1 OKRs

In FY24 we are dogfooding the new GitLab OKR tool. Since the tool is in it's early stages use the links below to access the OKRs for each group. New functionality is being release constantly and this manually generated list will likely become obsolete in the near future.

Links to OKRs

- Development Department
 - Ops Sub-Department
 - Configure
 - Release
 - Package
 - Verify
 - Pipeline Execution

- Pipeline Authoring
- Pipeline Insights
- Monitor and Runner
- Respond
- Observability
- Runner SaaS
- Runner

SECTION: engineering/devops/ops/deploy/_index.md

Who We Are

The Deploy stage is made up of one group:

1. Deploy:Environments

SECTION: engineering/devops/ops/deploy/environments/_index.md

Vision

For an understanding of where this team is going, take a look at the product vision.

As a member of the Ops Sub-department, you may also like to understand our overall vision.

Mission

OKRs

Product Indicators

Contribution to GitLab

Team Members

{{< team-by-manager-slug "nicolo-maria-mezzopera" >}}

Stable Counterparts

The following members of other functional teams are our stable counterparts:

{{< engineering/stable-counterparts role="Deploy:Environments" manager="nicolo-maria-mezzopera" >}}

Common Links

- General Slack channel
- Standup Slack channel
- Social Slack channel

Some dedicated Slack channels:

- Agent for Kubernetes: fagentforkubernetes
- Terraform backend: fterraformbackend
- Terraform provider: terraform-provider
- Auto DevOps: fautodevops
- Environments Detail Page: fenvironmentdetailspage

Insights

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="environments" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="environments" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="environments" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="environments" >}}
{{< /tableau >}}
```

Processes

Acronyms

- Engineers: All the Engineers of the Environments group
- Engineering: Engineers and the Engineering Manager
- EM: Engineering Manager
- PM: Product Manager
- FE: Frontend Engineer
- BE: Backend Engineer
- UX/PD: User Experience Designer
- TW: Technical Writer

Meetings

Environments Team Meeting

We have one team meeting each week. The purpose of this meeting is to share information about the ongoing projects. It also contains general announcements that are important for collaboration.

Meeting format:

- Before and during the meeting, team members write anything they want to verbalize in the notes attached to the meeting.
- We wait 1 or 2 minutes for the team members who want to join and then start recording when ready.
- During recording, we go over each point in the document.
- Anyone can facilitate the discussion. If the EM or PM is there, they will kick things off.
- If the author of the current point being discussed is available, they can verbalize their point.
- A short discussion may occur around each point, taking into consideration that we want to get through as much of the document as we can.
- All team members are welcome and encouraged to help take notes in the document while the discussion takes place.
- After we get through all the points, we stop recording.
- If there is left over time, team members may use the remaining unrecorded time to socialize or leave the meeting early.

If the meeting for the week has already taken place and you would like to add a new item for discussion, create a new section for the next meeting date above the last one and add your item.

Frontend, Go and Ruby Meetings

These are optional meetings on the team calendar. Everyone on the team is welcome. They are prioritized to be at a time where as many of the engineers who work primarily on the corresponding topics can attend.

These meetings are not too formal and also provide time for the engineers across time zones to discuss ongoing projects, ask questions, pair up, and catch up. We go through any agenda items first.

Meeting Links:

- Frontend: Frontend ChitChat
- Go: It's time to Go !!!
- Ruby: Ruby Chat

Technical Discovery Meetings

Sometimes we will encounter issues that need the input of the whole team to be refined and then worked on, such issues will be selected as a topic for a Technical Discovery meeting. We try to be conscious of sync time and so we expect a maximum of two of these meetings for

each milestone.

A technical discovery meeting consists of:

- One Meeting that everyone has a fair opportunity to join.
- The meeting is recorded.
- The meeting is announced at least one week before it will be held and each participant must familiarize themselves with the issue that is being investigated prior to attending.
- Discussing the topic async in advance in the issue/epic is encouraged.
- The PM will open the conversation either by describing the use case/scenario or by recording a quick video about it.
- The meeting is agenda first and everyone is expected to write their comments and questions in the agenda.
- If the agenda is empty the meeting is cancelled.
- The conversations in the meeting must be recorded in the same documents as notes.
- One host is decided for every meeting, and they are responsible to drive the conversation forward.
- In the last meeting someone is appointed to summarize the conversation either in the original issue or a technical document.

The goal of technical discovery meeting is to come up with a concrete technical proposal for the question at hand. We should not force a proposal, but aim to get there and write the conclusion accordingly with potential follow-ups.

Milestone Checkup Meeting

Twice a milestone, on Tuesday we hold a milestone check up meeting, where we either check the status of the work in progress issues or plan the next milestone.

Team issue tracker

- We use team issues tracker for internal discussions such as: milestone planning issues, events planning, etc...
- We also use a special project for Ops Section Async Updates.
- These issues are public by default, but can be confidential.

Issue refinement

During the Team Sync Meeting the PM will bring 2 to 3 issues that need a refinement DRI. During the meeting a DRI will be chosen who will be responsible for refining the issue. Team members are encouraged to refine issues that they create / stumble upon at any time during their work if they have the bandwidth to do so.

The refinement process is described in the issue template.

Labels used

Discovery backlog

In discovery we use the following labels

1. ~workflow::problem validation (optional) - to signal loosely defined problems where either the user problem or the business value is not yet understood
1. ~workflow::ready-for-design - this is our (likely endless) design backlog
1. ~workflow::solution validation (optional) - used for work with a concrete solution proposal that needs user validation
1. ~workflow::design - for ongoing design work

Delivery backlog

1. ~workflow::refinement - this is our delivery backlog; it contains all the issues that were not discussed by engineers in depth yet; this has no WIP limits
1. ~workflow::scheduling - this is our backlog of already discussed issues; these issues are still waiting to be scheduled or even to be put on the roadmap; issues entering should have a preliminary weight; this has no WIP limits
1. ~workflow::planning breakdown - this is the backlog for the upcoming milestone; it has a WIP limit of 2-months capacity
1. ~workflow::ready-for-development - this is the list of issues refined for the current or the upcoming milestone; it has a WIP limit of 2-months capacity; issues here should have a final weight

The PM is responsible for moving proposed issues from ~workflow::scheduling to ~workflow::planning breakdown and to maintain the WIP limits. Everyone is welcome to recommend issues for planning.

The EM is responsible for moving accepted issues from ~w